

Chapter 2

좌표계와 좌표변환

안테나가 켜 각도가 지도 위 위경도가 되기까지

측정

안테나

동체

NED

ECEF

LLA

레이다 시스템 소프트웨어 스테디 · Part 2

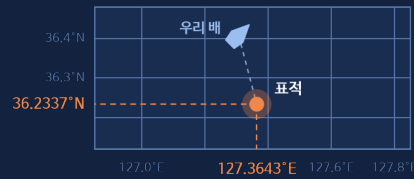
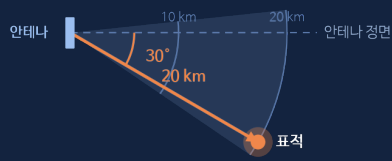
발표 대본

표지 · 약 20초

안녕하십니까. 이번 과제의 주제는 좌표계와 좌표변환입니다. 레이더 안테나가 켜 거리와 각도가 전시기의 위도 · 경도가 되기까지, 그 사이에 어떤 좌표계를 거치고 각 단계에서 무엇이 바뀌는지를 다루겠습니다.

진행은 개념, 설계, C 구현 검증 순서입니다.

레이다가 쟀 값과 전시기에 보일 값은 다르다



레이다(신호처리)가 주는 값

**"20 km 앞,
오른쪽으로 30도"**

안테나 자기 정면 기준

전시기에 보여야 하는 값

**"북위 36.2337도,
동경 127.3643도"**

지구 전체 기준

그 사이를 메우는 것이 좌표변환

안테나는 배가 어느 방향을 향하는지도, 지금 위도가 몇 도인지도 모른다

발표 대본

도입 · 약 45초

위 그림에서 왼쪽이 레이더 화면, 오른쪽이 전시기 지도입니다. 표적은 하나인데 한쪽은 부채꼴 위 한 점으로, 다른 쪽은 위경도 격자 위 한 점으로 찍힙니다.

같은 표적 하나를 두고, 레이더 신호처리가 주는 값은 "20 km 앞, 오른쪽으로 30도" 입니다. 안테나 자기 정면 기준입니다. 전시기에 보여야 하는 값은 "북위 36.2337도, 동경 127.3643도" 입니다. 지구 전체 기준입니다.

안테나가 아는 것은 전파 왕복 시간과 자기 정면 기준 두 각도뿐입니다. 배가 어느 방향을 향하는지도, 지금 위도가 몇 도인지도 모릅니다. 그 사이를 메우는 것이 좌표변환이고, 오늘 이야기의 전부입니다.

오늘 다룰 것

개념 → 설계 → 검증, 세 단계로 진행합니다

1

좌표계 여섯 가지

안테나 · 동체 · INS · NED/ENU
ECEF/ECI · LLA

각각 원점이 어디이고 축이
어디를 향하는지, 왜 필요한지

2

실전 설계

함선 고정형 안테나를 놓고
5단계 변환 경로를 설계

각 단계에서 무엇이 회전과
이동을 공급하는가

3

C 로 확인

설계한 변환을 구현하고
숫자로 검증

왕복 시험 · 확인 실험 3가지
UI 로 자세 바꿔 가며 확인

사전 지식은 삼각함수와 행렬 곱셈 정도면 충분합니다. 그마저도 필요한 곳에서 다시 설명합니다.

발표 대본

목차 · 약 30초

세 단계로 진행하겠습니다.

첫째, 좌표계 여섯 가지입니다. 안테나, 동체, INS, NED/ENU, ECEF/ECI, LLA 각각 원점이 어디이고 축이 어디를 향하는지, 왜 필요한지를 정리합니다.

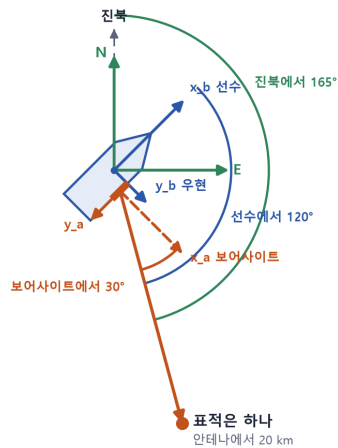
둘째, 실전 설계입니다. 함선에 고정된 안테나를 놓고 5단계 변환 경로를 설계하고, 각 단계에서 무엇이 회전과 이동을 공급하는지 봅니다.

셋째, C 로 확인입니다. 설계한 변환을 구현하고 왕복 시험과 확인 실험 세 가지로 숫자로 검증하고, UI 에서 자세를 바꿔 가며 확인합니다.

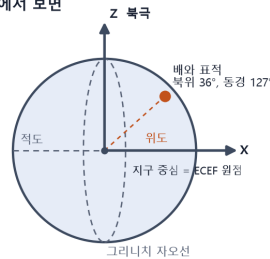
사전 지식은 삼각함수와 행렬 곱셈 정도면 충분하고, 그마저도 필요한 곳에서 다시 설명하겠습니다.

좌표계는 왜 여러 개나 필요할까

표적은 하나입니다. 어디에서(원점) 어느 방향을 기준으로(축) 재느냐에 따라 숫자만 달라집니다



지구에서 보면



ECEF : 지구 중심에서 잴 (X, Y, Z) m
LLA : 위도 · 경도 · 고도 (지도 좌표)

안테나 좌표계

원점 안테나 · 기준 방향 보어사이트
정면에서 오른쪽 30°, 20 000 m

동체 좌표계

원점 무게중심 · 기준 방향 선수
선수에서 오른쪽 120°, 20 015 m

NED 좌표계 (로컬)

원점 배 위치 · 기준 방향 진북
진북에서 165° → 남 19 340 m · 동 5 155 m

ECEF · LLA (지구)

원점 지구 중심 · 기준 적도면과 그리니치
북위 36.2337°, 동경 127.3643°

네 줄 모두 같은 점입니다. 숫자를 바꾸는 것은 원점과 축뿐이고, 그 둘을 맞춰 주는 계산이 좌표변환입니다.

발표 대본

개념 · 약 50초

그림의 표적은 하나입니다. 그런데 이 표적을 재는 기준이 여럿입니다. 안테나는 자기 위치에서 보어사이트를 기준으로 재고, 함선은 무게중심에서 선수를 기준으로 재고, 로컬 좌표계는 배 위치에서 진북을 기준으로 재고, 지구 좌표계는 지구 중심에서 적도면과 그리니치를 기준으로 잡니다.

그래서 같은 표적이 안테나에서는 "정면에서 오른쪽 30도, 20 000 m", 함선에서는 "선수에서 오른쪽 120도, 20 015 m", 로컬에서는 "진북에서 165도, 남쪽으로 19 340 m 동쪽으로 5 155 m", 지도에서는 "북위 36.2337도, 동경 127.3643도" 가 됩니다. 각도가 30, 120, 165로 달라진 것은 기준 방향, 즉 축이 다르기 때문이고, 거리가 20 000 m와 20 015 m로 조금 다른 것은 기준점, 즉 원점이 다르기 때문입니다.

네 줄 모두 같은 점입니다. 숫자를 바꾸는 것은 원점과 축뿐이고, 그 둘을 맞춰 주는 계산이 좌표변환입니다.

좌표계를 읽는 세 가지 질문

새 좌표계를 만나면 이 셋만 확인하면 됩니다

①

원점이 어디인가?

"0, 0, 0" 이 물리적으로 어느 지점인가.
평행이동 계산의 기준이 된다.

②

축이 어디를 향하는가?

특히 z 가 위인지 아래인지.
 x 앞 · y 오른쪽으로 잡으면 z 는 저절로 아래가 된다.
항법 분야의 이 관습, 처음엔 거의 반드시 헷갈린다.

③

무엇에 붙어 있는가?

배에 붙어 있으면 배가 흔들릴 때
좌표계도 같이 흔들린다.
표적이 가만히 있어도 좌표값이 변한다.

이 발표의 약속 : 안테나·동체·NED 좌표계에서 z 는 아래. "10 m 위" 는 $z = -10$ 입니다.

발표 대본

개념 · 약 45초

새 좌표계를 만나면 세 가지만 확인하면 됩니다.

첫째, 원점이 어디인가. (0, 0, 0) 이 물리적으로 어느 지점인지입니다. 평행이동의 기준이 됩니다.

둘째, 축이 어디를 향하는가. 특히 z 가 위인지 아래인지입니다. x 를 앞, y 를 오른쪽으로 잡으면 오른손 좌표계에서 z 는 저절로 아래가 됩니다. 항법 분야의 이 관습이 처음엔 거의 반드시 헷갈립니다.

셋째, 무엇에 붙어 있는가. 배에 붙어 있으면 배가 흔들릴 때 좌표계도 같이 흔들립니다. 표적이 가만히 있어도 좌표값이 변합니다.

이 발표의 약속 하나만 기억해 주십시오. 안테나, 동체, NED 좌표계에서 z 는 아래입니다. "10 m 위" 는 $z = -10$ 입니다. 지구 중심 좌표계 ECEF 만 Z 가 북극을 향합니다.

변환은 두 동작뿐

복잡한 수학처럼 들리지만 실제로 하는 일은 두 가지입니다

1 회전 (rotation)

기준 방향이 다를 때

"내 정면"과 "배의 정면"이 다르면
회전이 필요하다.

안테나가 우현 90도를 보고 있으니,
안테나의 "앞"은 배의 "오른쪽"이다.

2 평행이동 (translation)

기준 점이 다를 때

안테나가 무게중심에서
30 m 뒤 · 10 m 위에 있으면
그만큼 옮겨줘야 한다.

이 오프셋을 레버암이라 부른다.

모든 단계는 이 둘 중 하나거나, 둘 다입니다. 그게 전부입니다.

$$p_{\text{함선}} = \underbrace{Rz(90^\circ)}_{\text{회전}} \times \underbrace{p_{\text{안테나}}}_{\text{평행이동}} + \underbrace{(-30, 0, -10)}_{\text{평행이동}} \text{ m}$$

$Rz(90^\circ)$ 는 z 축(위아래 축) 둘레로 90° 돌리는 회전행렬입니다. 생김새는 다음 두 장에서 나옵니다.

발표 대본

개념 · 약 40초

복잡한 수학처럼 들리지만 실제로 하는 일은 두 가지입니다.

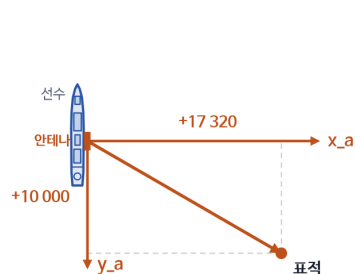
하나는 회전입니다. 기준 방향이 다를 때 필요합니다. 안테나가 우현 90도를 보고 있으니 안테나의 "앞"은 배의 "오른쪽"입니다.

다른 하나는 평행이동입니다. 기준점이 다를 때 필요합니다. 안테나가 무게중심에서 30 m 뒤, 10 m 위에 있으면 그만큼 옮겨줘야 합니다. 이 오프셋을 레버암이라고 부릅니다.

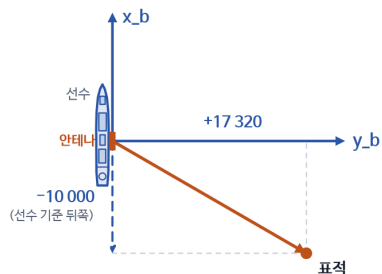
아래 식이 그 둘을 한 줄로 쓴 것입니다. 함선 좌표 = $Rz(90^\circ)$ 곱하기 안테나 좌표, 더하기 $(-30, 0, -10)$. $Rz(90^\circ)$ 는 z 축, 즉 위아래 축 둘레로 90도 돌리는 회전행렬인데 생김새는 다음 두 장에서 나옵니다. 모든 단계는 이 둘 중 하나거나 둘 다입니다. 그게 전부입니다.

회전 — 표적이 아니라 축이 돌아간다

(a) 안테나 축에서 읽으면

 x_a = 보어사이트 (우현 쪽), y_a = 그 오른쪽 (선미 쪽)읽은 값: $x_a = +17\,320\text{ m}$, $y_a = +10\,000\text{ m}$

(b) 함선 축에서 읽으면

 x_b = 선수, y_b = 우현 (x_a 와 y_b 는 같은 방향)읽은 값: $x_b = -10\,000\text{ m}$, $y_b = +17\,320\text{ m}$

배도 표적도 그대로다. 축만 90° 돌아가 있다.

안테나의 "정면"이 함선의 "우현"이라서, 같은 표적이 $(+17\,320, +10\,000) \rightarrow (-10\,000, +17\,320)$

두 숫자가 자리를 바꾸고 부호 하나가 뒤집혔다. 이것이 90도 회전의 정체입니다.

발표 대본

개념 · 약 35초

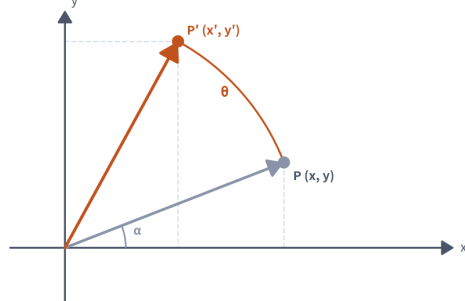
회전에서 가장 헷갈리는 부분입니다. 두 그림에서 배도 표적도 같은 자리에 있습니다. 달라진 것은 축뿐입니다.

왼쪽은 안테나의 축에서, 오른쪽은 함선의 축에서 같은 화살표를 읽은 값입니다. 안테나의 정면이 함선의 우현이라서, 같은 표적이 $(+17,320, +10,000)$ 에서 $(-10,000, +17,320)$ 이 됩니다. 두 숫자가 자리를 바꾸고 부호 하나가 뒤집혔습니다. 이것이 90도 회전의 정체입니다. 표적을 돌린 것이 아니라 90도 어긋난 축에서 다시 읽은 것입니다.

회전행렬은 어디서 나오는가

외울 것이 아니라 삼각함수 덧셈정리 한 줄에서 바로 나옵니다

회전행렬은 어디서 나오는가 — 2차원 회전 유도



원래 점을 극좌표로 쓰면
 $x = r \cdot \cos \alpha$, $y = r \cdot \sin \alpha$

θ 만큼 더 돌리면 각이 $\alpha + \theta$ 가 되므로
 $x' = r \cdot \cos(\alpha + \theta) = r \cdot \cos \alpha \cdot \cos \theta - r \cdot \sin \alpha \cdot \sin \theta$
 $y' = r \cdot \sin(\alpha + \theta) = r \cdot \sin \alpha \cdot \cos \theta + r \cdot \cos \alpha \cdot \sin \theta$

$r \cdot \cos \alpha = x$, $r \cdot \sin \alpha = y$ 를 되돌려 넣으면

$x' = x \cdot \cos \theta - y \cdot \sin \theta$

$y' = x \cdot \sin \theta + y \cdot \cos \theta$

이 두 줄을 표로 적은 것이 $R_z(\theta)$ 다.

그래서 $R_z(\theta)$ 는 이렇게 생겼다

$$\begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

회전행렬이 늘 갖는 세 성질

- 행과 열의 길이가 1 이고 서로 직교(90°)한다
- $\det = +1$ — 크기를 안 바꾸고 거울상도 아니다
- 역행렬 = 전치(행·열 맞바꾸기, $R^{-1} = R^T$) — 되돌리기가 공짜다

행렬을 쓰는 이유 : ① 세 축을 한 번에 처리 ② 여러 회전을 곱해서 하나로 합침 ③ 되돌아갈 땐 전치만 하면 됨

발표 대본

개념 · 약 50초

회전행렬은 외울 것이 아니라 삼각함수 덧셈정리 한 줄에서 바로 나옵니다. 각도 θ 만큼 돌린 축에서 좌표를 다시 읽으면 $\cos$ 과 $\sin$ 이 이렇게 배치된 3×3 행렬 $R_z(\theta)$ 가 됩니다.

회전행렬은 늘 세 가지 성질을 갖습니다. 각 행과 열의 길이가 1 이고 서로 직교, 즉 90° 를 이룬다. 행렬식이 +1 이라서 크기를 바꾸지도 거울상으로 뒤집지도 않는다. 그리고 역행렬이 전치, 즉 행과 열을 맞바꾼 것과 같다. 마지막 성질 덕분에 되돌리기가 공짜입니다.

행렬을 쓰는 이유는 세 가지입니다. 세 축을 한 번에 처리하고, 여러 회전을 곱해서 하나로 합치고, 되돌아갈 때는 전치만 하면 됩니다. 구현에서도 정확히 이 세 가지만 씁니다.

Part 1

좌표계 여섯 가지

각각 원점이 어디이고, 축이 어디를 향하고, 무엇에 붙어 있는가

측정

>

안테나

>

동체

>

NED

>

ECEF

>

LLA

발표 대본

섹션 · 약 15초

첫 번째 주제입니다. 여섯 가지 좌표계를 측정에서 지도까지 순서대로 보겠습니다. 각각 원점이 어디이고, 축이 어디를 향하고, 무엇에 붙어 있는지를 기준으로 봅니다.

안테나 좌표계 ① 극좌표 (Polar)

레이다가 물리적으로 잴 수 있는 것 — 거리 하나와 각도 둘

측정

안테나

동체

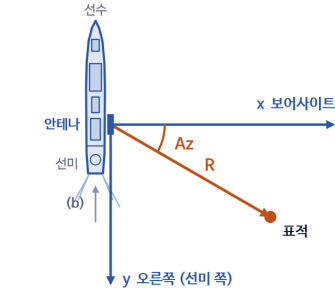
NED

ECEF

LLA

(a) 위에서 본 그림 — 방위각 Az

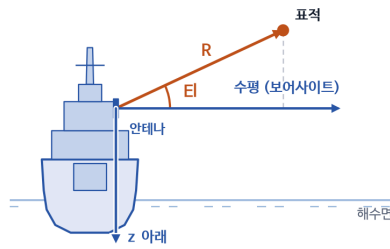
갑판을 내려다본 모습. 보어사이트에서 오른쪽으로 잰 각



(b) 는 이 배를 화살표 방향으로 본 모습이다.

(b) 뒤에서 본 그림 — 고각 θ

같은 배를 선미 쪽에서 본 모습. 수평면에서 위로 잰 각



z 는 아래가 +, 표적이 수평면보다 위에 있으면 z 는 음수가 된다.

세 성분

R 시선거리

전파 왕복시간 $\times c \div 2$

Az 방위각

보어사이트에서 오른쪽으로

θ 고각

수평면에서 위로

20 km / 30° / 0°

보어사이트 = 안테나가 똑바로 바라보는 방향. 손전등으로 치면 빛이 가장 센 한가운데 방향입니다.

발표 대본

좌표계 · 약 40초

레이다가 물리적으로 잴 수 있는 것은 거리 하나와 각도 둘입니다.

R 은 시선거리로 전파 왕복시간 곱하기 광속 나누기 2 입니다. Az 는 방위각으로 보어사이트에서 오른쪽으로 잰 각, θ 은 고각으로 수평면에서 위로 잰 각입니다. 과제 예제값이 20 km, 30도, 0도입니다.

보어사이트는 안테나가 똑바로 바라보는 방향입니다. 손전등으로 치면 빛이 가장 센 한가운데 방향입니다.

안테나 좌표계 ② 직교좌표 (Cartesian)

회전행렬을 곱하려면 x-y-z 세 성분으로 풀어써야 합니다

측정

안테나

동체

NED

ECEF

LLA

$$\begin{aligned} x &= R \times \cos(EI) \times \cos(Az) && \leftarrow \text{보어사이트 방향 성분} \\ y &= R \times \cos(EI) \times \sin(Az) && \leftarrow \text{오른쪽 방향 성분} \\ z &= -R \times \sin(EI) && \leftarrow \text{아래 방향 성분 (위로 향하면 음수)} \end{aligned}$$

예제 값을 넣으면

$$\begin{aligned} x &= 20000 \times 1 \times \cos 30^\circ = +17\,320.5081 \text{ m} \\ y &= 20000 \times 1 \times \sin 30^\circ = +10\,000.0000 \text{ m} \\ z &= -20000 \times \sin 0^\circ = 0.0000 \text{ m} \end{aligned}$$

왜 직교좌표로 바꾸나

극좌표는 사람이 읽기 좋지만, 극좌표끼리는 회전을 합성할 수 없다.

회전행렬은 벡터에만 곱할 수 있으므로 각도로 표현된 방향을 세 성분으로 풀어쓰는 준비 단계다.

역변환에서는 asin 대신 atan2 를 쓴다.
반올림으로 NaN 이 나타나 $\pm 90^\circ$ 근처에서 정밀도가 무너지는 걸 막기 위해서다.

읽는 법 : $\cos(EI)$ 로 표적을 먼저 수평면에 눕히고, 그 길이를 Az 로 앞(x)·오른쪽(y)에 나눠 담습니다.
위아래(z)는 $\sin(EI)$ 몫 — z 는 아래가 양수라, 표적이 위에 있으면 z 가 음수가 됩니다.

발표 대본

좌표계 · 약 55초

회전행렬을 곱하려면 x, y, z 세 성분으로 풀어써야 합니다. x 는 $R \cos(EI) \cos(Az)$ 로 보어사이트 방향 성분, y 는 $R \cos(EI) \sin(Az)$ 로 오른쪽 성분, z 는 $-R \sin(EI)$ 로 아래 방향 성분입니다.

예제값을 넣으면 x 는 +17,320.5081 m, y 는 +10,000 m, z 는 0 입니다.

왜 직교좌표로 바꾸느냐 하면, 극좌표끼리는 회전을 합성할 수 없기 때문입니다. 회전행렬은 벡터에만 곱할 수 있으니, 각도로 표현된 방향을 세 성분으로 풀어쓰는 준비 단계입니다.

역변환에서는 asin 대신 atan2 를 씁니다. 반올림으로 인자가 1 을 살짝 넘으면 asin 은 NaN 을 내고, $\pm 90^\circ$ 도 근처에서 정밀도가 무너지기 때문입니다. 식을 읽는 법은 이렇습니다. $\cos(EI)$ 로 표적을 먼저 수평면에 눕히고, 그 길이를 Az 로 앞과 오른쪽에 나눠 담습니다. 위아래는 $\sin(EI)$ 몫이고, z 축은 아래가 양수라서 표적이 위에 있으면 z 가 음수가 됩니다.

안테나 좌표계 ③ UV (방향코사인)

표적 방향을 안테나 면에 드려온 그림자로 표현합니다



왜 UV 를 쓰나

현대 레이다는 대부분 위상배열이다. 수백~수천 개 소자에 위상차를 줘서 빔 방향을 전자적으로 바꾼다.

$$\psi = -k(m \cdot d \cdot u_0 + n \cdot d \cdot v_0)$$

소자 번호에 u, v 를 곱하기만 하면 된다.
각도(Az, EI)로는 이렇게 단순해지지 않는다.
되돌릴 때는 $asin$ 대신 $atan2$ 를 쓴다.
 $Az = atan2(u, w)$
 $EI = atan2(v, \sqrt{u^2 + w^2})$

빔 굽기가 조향각과 무관하게 일정하고, 격자로브 위치도 $u_0 \pm \lambda/d$ 로 단순하게 나옵니다.
UV 는 빔 조향용 표현이라 이번 변환 체인(극좌표 → 직교 → 동체 → ...)에는 들어가지 않습니다.

발표 대본

좌표계 · 약 40초

표적 방향을 안테나 면에 드려온 그림자로 표현한 것이 UV, 방향코사인입니다.

UV 를 쓰는 이유는 현대 레이다가 대부분 위상배열이기 때문입니다. 수백에서 수천 개 소자에 위상차를 줘서 빔 방향을 전자적으로 바꾸는데, 그 위상이 소자 번호에 u, v 를 곱하기만 하면 나옵니다. 각도 Az, EI 로는 이렇게 단순해지지 않습니다.

빔 굽기가 조향각과 무관하게 일정하고 격자로브 위치도 단순하게 나옵니다. UV 에서 각도로 되돌릴 때도 $asin$ 대신 $atan2$ 를 씁니다. 다만 이것은 빔 조향, 즉 안테나 내부의 일이라서 이번 데이터 처리 체인에는 들어가지 않습니다.

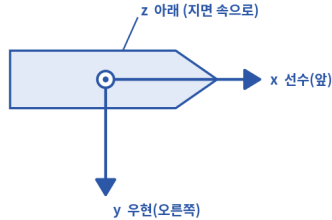
동체(Body) 좌표계

배에 실린 모든 장비의 공통 기준 — 안테나-INS-무장이 여기로 모입니다



동체(Body) 좌표계 — FRD

위에서 본 그림. 원점은 무게중심.



자세각 세 가지

$C_{ned \leftarrow body} = R_z(yaw) \cdot R_y(pitch) \cdot R_x(roll)$ (3-2-1 순서)



원점

배의 무게중심

축

x 선수(앞) · y 우현(오른쪽) · z 아래
→ FRD

자세각

roll 좌우 기울기 · pitch 앞뒤 기울기
· yaw 선수 방위

합치는 순서

3-2-1 (yaw → pitch → roll)

발표 대본

좌표계 · 약 45초

배에 실린 모든 장비의 공통 기준입니다. 안테나, INS, 무장이 여기로 모입니다.

원점은 배의 무게중심, 축은 x가 선수, y가 우현, z가 아래라서 FRD라고 부릅니다. 자세각은 roll이 좌우 기울기, pitch가 앞뒤 기울기, yaw가 선수 방위입니다.

주의할 점은 합치는 순서입니다. 회전은 곱셈 순서를 바꾸면 결과가 달라집니다. 그래서 yaw, pitch, roll 순서인 3-2-1을 문서로 고정해야 합니다. 구현에서도 $R_z(yaw) \cdot R_y(pitch) \cdot R_x(roll)$ 순서로 곱합니다.

INS 좌표계 ① INS 는 무엇을 재는가

관성항법장치 — 회전과 이동에 필요한 값을 공급합니다

측정

안테나

동체

NED

ECEF

LLA

자이로 × 3

각속도 — 초당 몇 도씩 돌고 있는가
→ 시간에 대해 적분하면 자세 (roll-pitch-yaw)

가속도계 × 3

비력 — 센서가 느끼는 가속도 (정지 중에도 1 g 잡힘)
→ 중력 모델을 빼고 두 번 적분하면 속도와 위치

핵심 : INS 는 바깥 세상을 전혀 보지 않는다

GPS 처럼 위성 신호를 받지도, 지형을 관측하지도 않는다. 출발점만 알려주면 자기가 느낀 각속도와 가속도를 계속 더해 나간다.

장점

- 외부 신호가 필요 없다 — 전파방해에 강하다
- 갱신 속도가 매우 빠르다 (보통 100 Hz 이상)

단점

- 오차가 시간이 갈수록 쌓인다 (적분하므로)
- 그래서 GPS 같은 외부 기준과 결합해 잡아준다

출항 전 초기 정렬 — 중력으로 수평을 잡고(leveling), 지구 자전을 감지해 북쪽을 찾는다(gyrocompassing)

발표 대본

좌표계 · 약 50초

관성항법장치, INS 는 회전과 이동에 필요한 값을 공급합니다.

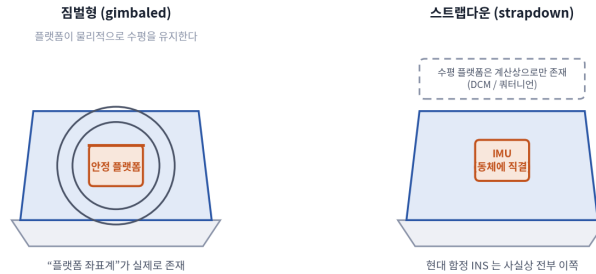
자이로 세 개가 각속도, 즉 초당 몇 도씩 돌고 있는지를 재고, 시간에 대해 적분하면 자세가 나옵니다. 가속도계 세 개는 비력, 즉 센서가 느끼는 가속도를 잽니다. 가만히 있어도 중력 1 g 가 잡히기 때문에 중력 모델을 뺀 뒤 두 번 적분하면 속도와 위치가 나옵니다.

핵심은 INS 가 바깥 세상을 전혀 보지 않는다는 것입니다. GPS 처럼 위성 신호를 받지도 지형을 관측하지도 않고, 출발점만 알려주면 자기가 느낀 각속도와 가속도를 계속 더해 나갑니다. 그래서 외부 신호가 필요 없어 전파방해에 강하고 갱신이 100 Hz 이상으로 빠르지만, 오차가 시간이 갈수록 쌓입니다. 그래서 GPS 같은 외부 기준과 결합해서 잡아줍니다.

출항 전 초기 정렬에서 중력으로 수평을 잡고, 지구 자전을 감지해서 북쪽을 찾습니다.

INS 좌표계 ② 스트랩다운과 짐벌형

"플랫폼 좌표계"라는 말의 의미가 여기서 갈립니다



	짐벌형 (gimbaled)	스트랩다운 (strapdown)
구조	센서가 짐벌 위에 얹혀 있고 모터가 늘 수평 유지	센서가 동체에 직접 고정, 배와 함께 기울다
수평 유지	기계가 물리적으로 한다	계산이 한다 (DCM / 쿼터니언)
"플랫폼 좌표계"	실제로 존재하는 물리적 프레임	소프트웨어 안의 상태값일 뿐
특징	정밀하지만 크고 무겁고 비싸다	현대 함정 INS는 사실상 전부 이쪽

발표 대본

좌표계 · 약 40초

"플랫폼 좌표계" 라는 말의 의미가 여기서 갈립니다.

짐벌형은 센서가 짐벌 위에 얹혀 있고 모터가 늘 수평을 유지합니다. 플랫폼 좌표계가 실제로 존재하는 물리적 프레임입니다. 정밀하지만 크고 무겁고 비쌉니다.

스트랩다운은 센서가 동체에 직접 고정되어 배와 함께 기울고, 수평 유지는 계산이 합니다. 플랫폼 좌표계는 소프트웨어 안의 상태값일 뿐입니다. 현대 함정 INS는 사실상 전부 이쪽입니다.

INS 좌표계 ③ 이번 설계에서는

조건 하나가 복잡한 이야기를 통째로 없애줍니다

측정

안테나

동체

NED

ECEF

LLA

"INS 는 플랫폼 무게중심과 축이 일치한 상태로 설치되어 있음"

INS 좌표계 = 동체(Body) 좌표계 (설치 오차 0, 레버암 0)

따라서 INS 를 독립된 변환 단계로 두지 않습니다. 좌표계가 아니라 값의 공급원으로만 씁니다.

INS 가 주는 값	어디에 쓰이는가
roll, pitch, yaw	3단계 회전행렬 $C_{ned \leftarrow body}$
위도, 경도, 고도	4단계 로컬→ECEF 회전과 평행이동
(속도)	표적 추적 필터 · 도플러 보정 — 이 발표 범위 밖

발표 대본

좌표계 · 약 40초

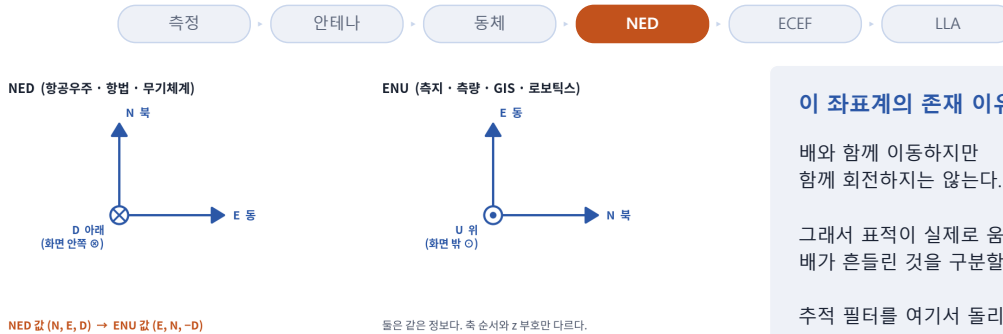
과제 조건 하나가 복잡한 이야기를 통째로 없애줍니다. "INS 는 플랫폼 무게중심과 축이 일치한 상태로 설치되어 있음" 입니다.

따라서 INS 좌표계는 동체 좌표계와 같습니다. 설치 오차 0, 레버암 0 입니다. 그래서 INS 를 독립된 변환 단계로 두지 않고, 좌표계가 아니라 값의 공급원으로만 씁니다.

INS 가 주는 roll, pitch, yaw 는 3단계 동체에서 NED 로 가는 회전행렬에, 위도 · 경도 · 고도는 4단계 로컬에서 ECEF 로 가는 회전과 평행이동에 씁니다. 속도는 추적 필터나 도플러 보정에 쓰이는데 이번 발표 범위 밖입니다.

NED / ENU 국지수평 좌표계

배가 아무리 흔들려도 북쪽은 계속 북쪽입니다



이 좌표계의 존재 이유

배와 함께 이동하지만
함께 회전하지는 않는다.

그래서 표적이 실제로 움직인 것과
배가 흔들린 것을 구분할 수 있다.

추적 필터를 여기서 돌리는 이유다.

"아래"는 지구 중심 방향이 아니다

타원체 법선의 반대 방향이다. 지구 중심 방향과는
위도 45° 부근에서 최대 약 0.19° 어긋나는데,
이 둘을 혼동하면 지표 위치가 약 21 km 틀어진다.

로컬 평면의 한계 — 지구는 둥글다

원점에서 멀어질수록 실제 지표면이 평면 아래로 내려간다
5 km 2 m · 20 km 31 m · 50 km 196 m · 100 km 786 m

발표 대본

좌표계 · 약 50초

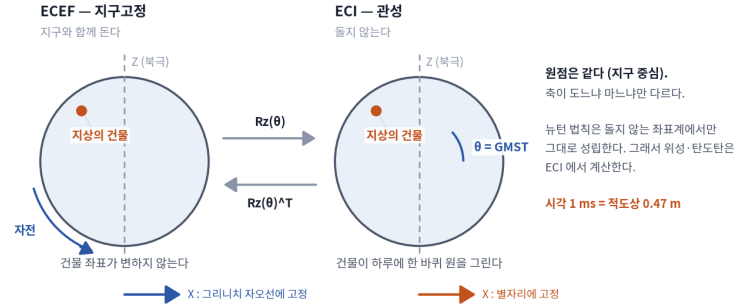
배가 아무리 흔들려도 북쪽은 계속 북쪽입니다. 이 좌표계의 존재 이유가 그것입니다. 배와 함께 이동하지만 함께 회전하지는 않습니다. 그래서 표적이 실제로 움직인 것과 배가 흔들린 것을 구분할 수 있고, 추적 필터를 여기서 돌립니다.

두 가지 주의점이 있습니다. 첫째, "아래"는 지구 중심 방향이 아니라 타원체 법선의 반대 방향입니다. 지구 중심 방향과는 위도 45도 부근에서 최대 약 0.19도 어긋나는데, 이 둘을 혼동하면 지표 위치가 약 21 km 틀어집니다.

둘째, 로컬 평면의 한계입니다. 지구는 둥글어서 원점에서 멀어질수록 실제 지표면이 평면 아래로 내려갑니다. 5 km에서 2 m, 20 km에서 31 m, 50 km에서 196 m, 100 km에서 786 m입니다. 이 숫자는 뒤에서 다시 나옵니다.

ECEF / ECI 지구 중심 좌표계

원점은 같고, 축이 도느냐 마느냐만 다릅니다



ECEF 는 왜 필요한가

로컬 좌표계에서 위경도로 곧장 갈 수 없다.
로컬 원점(배 위치)이 지구 어디인지는
지구 기준 좌표로만 표현되기 때문이다. → 다리 역할

ECI 는 이번 설계에 필요 없다

뉴턴 법칙은 회전하지 않는 좌표계에서만 그대로 성립.
그래서 중력만 받고 날아가는 물체(위성·탄도탄)는 ECI 에서 계산.
항공기·수상함 표적은 ECEF/NED 로 충분하다.

발표 대본

좌표계 · 약 40초

둘 다 원점은 지구 중심이고, 축이 지구와 함께 도느냐 마느냐만 다릅니다.

ECEF 가 필요한 이유는 로컬 좌표계에서 위경도로 곧장 갈 수 없기 때문입니다. 로컬 원점, 즉 배의 위치가 지구 어디인지는 지구 기준 좌표로만 표현됩니다. 다리 역할입니다.

ECI 는 이번 설계에 필요 없습니다. 뉴턴 법칙은 회전하지 않는 좌표계에서만 그대로 성립하므로, 중력만 받고 날아가는 위성이나 탄도탄은 ECI 에서 계산합니다. 항공기나 수상함 표적은 ECEF 와 NED 로 충분합니다.

LLA 측지 좌표계

우리가 아는 그 지도 좌표 — 다만 지구는 구가 아닙니다

측정

안테나

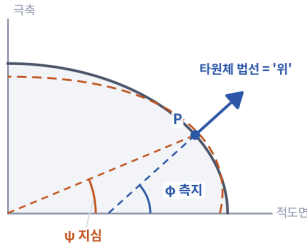
동체

NED

ECEF

LLA

지구 타원체 · 측지위도 · 고도 기준 (편평률을 크게 과장한 그림)



두 위도의 차이

$\tan \psi = (1 - e^2) \cdot \tan \phi$, 최대 0.192°
→ 지표에서 약 21 km 에 해당

고도의 두 기준

— 타원체 (WGS-84) — LLA 의 고도 기준
— 지오이드 — 해도 · 고도계의 '해발' 기준

해발고도 = 타원체고 - 지오이드고
한국 근해 지오이드고 $\approx +20 \sim +25$ m

시스템 내부는 타원체고로 통일하고,
표시할 때만 변환하는 것이 안전하다.

WGS-84 타원체

a (장반경) 6 378 137.0 m

b (단반경) 6 356 752.3 m

1/f 298.257223563

적도 반지름과 극 반지름의 차이가
약 21 km. 지구 크기에 비하면 0.3 %
정도지만, 구로 계산하면 위치가
20 km 넘게 어긋난다 (위도 45° 부근).

GPS 와 해도가 쓰는 것도 이것

고도의 함정 : LLA 의 고도는 타원체고 (HAE) — 해도·고도계의 "해발" 과 다릅니다

해발고도 = 타원체고 - 지오이드고. 한국 근해 지오이드고는 약 +20 ~ +25 m. 시스템 내부는 하나로 통일할 것.

발표 대본

좌표계 · 약 50초

우리가 아는 그 지도 좌표입니다. 다만 지구는 구가 아닙니다.

WGS-84 타원체는 장반경 6,378,137.0 m, 단반경 6,356,752.3 m, 편평률의 역수 298.257223563 입니다. 적도 반지름과 극 반지름의 차이가 약 21 km 로, 지구 크기에 비하면 0.3 % 정도지만 구로 계산하면 위도 45도 부근에서 위치가 20 km 넘게 어긋납니다. GPS 와 해도가 쓰는 것도 이것입니다.

고도에 함정이 하나 있습니다. LLA 의 고도는 타원체고, HAE 이고 해도나 고도계의 "해발" 과 다릅니다. 해발고도는 타원체고에서 지오이드고를 뺀 값이고, 한국 근해 지오이드고는 약 +20 에서 +25 m 입니다. 시스템 내부는 하나로 통일해야 합니다.

여섯 좌표계 한눈에

좌표계	원점	축	붙은 곳	주 용도
안테나	안테나 면 중심	x 보어사이트 · y 오른쪽 · z 아래	배	표적 측정 (R, Az, El) / 빔 조향 (u, v)
동체	배 무게중심	x 선수 · y 우현 · z 아래	배	탑재 장비의 공통 기준
INS	INS 설치 위치	자이로·가속도계 3축	배	위치 · 자세 · 속도 공급
NED / ENU	배의 현재 위치	N,E,D / E,N,U	바다 위 지점 (회전 안 함)	데이터 처리 (추적 · 필터링)
ECEF	지구 중심	Z 북극 · X 적도 · Y 그리니치	지구 (자전함)	지구 기준 공통 좌표, 위경도로 가는 다리
ECI	지구 중심	별자리에 고정	없음 (회전 안 함)	위성 · 탄도 표적의 운동 계산
LLA	(적도면 / 그리니치)	위도 · 경도 · 고도	지구	지도 표시, 외부 전달

순서를 건너뛸 수 없습니다. 안테나는 배에 붙어 있고, 배의 방향은 북쪽 기준으로 알려지고, 로컬 원점의 위치는 지구 좌표로만 표현되기 때문입니다.

발표 대본

정리 · 약 40초

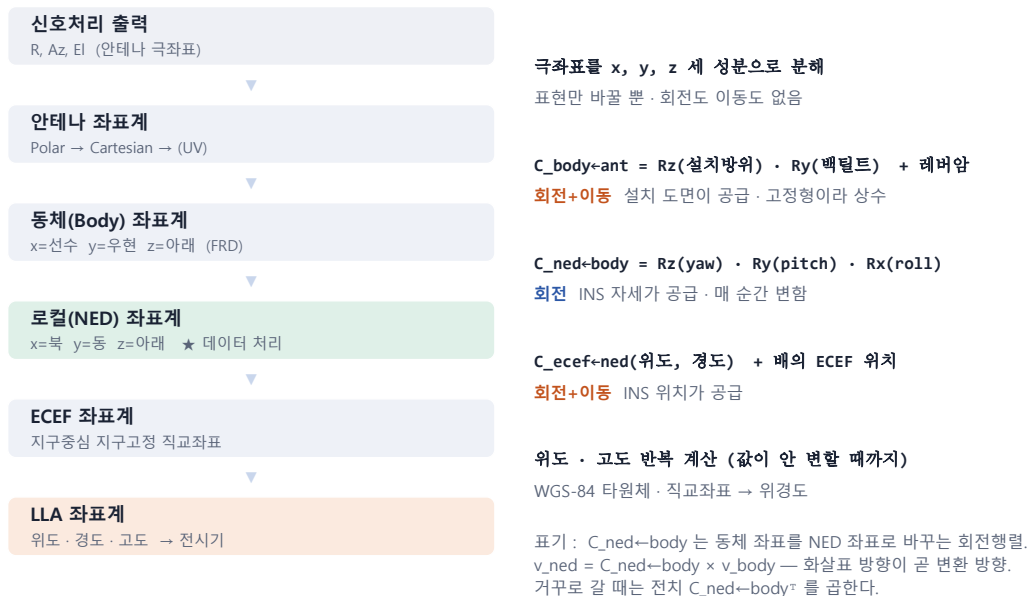
여섯 좌표계를 표 하나로 모았습니다. 원점, 축, 붙은 곳, 주 용도 순서입니다.

안테나와 동체와 INS 는 배에 붙어 있고, NED 는 바다 위 지점에 붙어 회전하지 않고, ECEF 는 지구에 붙어 자전하고, ECI 는 아무 데도 안 붙어 있고, LLA 는 지구 기준입니다.

중요한 것은 순서를 건너뛸 수 없다는 점입니다. 안테나는 배에 붙어 있고, 배의 방향은 북쪽 기준으로 알려지고, 로컬 원점의 위치는 지구 좌표로만 표현되기 때문입니다.

변환 체인 — 무엇이 회전과 이동을 공급하는가

회전은 설치 방위와 INS(자세·위경도)가, 이동은 레버암과 INS(배 위치)가 공급합니다. 이 구분이 그대로 오차의 출처 목록이 됩니다.



발표 대본

설계 · 약 60초

전체 체인을 한 장으로 보겠습니다. 왼쪽이 좌표계 순서, 오른쪽이 각 단계의 변환입니다.

신호처리 출력 R, Az, El 이 안테나 극좌표입니다. 1단계에서 이것을 x, y, z 세 성분으로 분해합니다. 표현만 바꿀 뿐 회전도 이동도 없습니다.

2단계 안테나에서 동체로는 $R_z(\text{설치방위})$ 곱하기 $R_y(\text{백틸트})$ 로 회전하고 레버암을 더합니다. 설치 도면이 공급하고, 고정형이라 상수입니다.

3단계 동체에서 NED 로는 $R_z(\text{yaw}) \cdot R_y(\text{pitch}) \cdot R_x(\text{roll})$ 로 회전만 합니다. INS 자세가 공급하고 매 순간 변합니다. 여기가 데이터 처리 구간입니다.

4단계 NED 에서 ECEF 로는 위도 · 경도로 정해지는 회전에 배의 ECEF 위치를 더합니다. INS 위치가 공급합니다.

5단계 ECEF 에서 LLA 로는 위도와 고도를 번갈아 계산하는 반복입니다. WGS-84 타원체 기준입니다.

회전은 설치 방위와 INS 의 자세 · 위경도가, 이동은 레버암과 INS 의 배 위치가 공급합니다. 이 구분이 그대로 오차의 출처 목록이 됩니다. 표기를 하나 짚고 가면, $C_{ned \leftarrow body}$ 는 동체 좌표를 NED 좌표로 바꾸는 회전행렬이고 화살표 방향이 곧 변환 방향입니다. 거꾸로 갈 때는 전치를 곱합니다.

Part 2

실전 설계

함선 고정형 안테나를 놓고 변환 경로를 설계합니다

신호처리 → 데이터 처리 (좌표변환 + 추적) → 통제제어 전시기

발표 대본

섹션 · 약 15초

두 번째 주제입니다. 함선 고정형 안테나를 놓고 변환 경로를 설계합니다. 흐름은 신호처리에서 데이터 처리, 즉 좌표변환과 추적을 거쳐 통제제어 전시기로 갑니다.

상황

주어진 조건 여섯 가지와, 그것이 설계를 어떻게 묶는가

조건	내용	설계에 미치는 영향
① 플랫폼	해상 함선, 고정형 안테나	안테나가 돌지 않는다 → 회전행렬이 상수. 대신 배의 흔들림을 전부 계산으로 보상
② 안테나 설치	선수 기준 시계방향 90°, 무게중심에서 30 m 뒤 / 10 m 위	설치 방위 90°, 백틸트 0°, 레버암 (-30, 0, -10) m
③ INS 설치	무게중심과 축 일치	INS 좌표계 = 동체 좌표계 → 변환 단계 하나가 통째로 생략
④ 입력	안테나 기준 거리·방위각·고각	체인의 출발점이 안테나 극좌표로 확정
⑤ 처리	로컬 좌표계에서 수행	체인의 중간 목적지가 NED 로 확정
⑥ 출력	위/경/고도 + 안테나 극좌표	정변환뿐 아니라 역변환도 필요

④·⑤·⑥ 이 체인의 출발점·중간점·도착점을 이미 지정합니다. 설계자가 정할 것은 그 사이를 어떻게 이을 것인가입니다.

발표 대본

설계 · 약 60초

과제 조건 여섯 가지와 그것이 설계를 어떻게 묶는지입니다.

조건 1, 해상 함선에 고정형 안테나입니다. 안테나가 돌지 않으니 회전행렬이 상수이고, 대신 배의 흔들림을 전부 계산으로 보상해야 합니다.

조건 2, 안테나는 선수 기준 시계방향 90도, 무게중심에서 30 m 뒤, 10 m 위입니다. 설치 방위 90도, 백틸트 0도, 레버암 (-30, 0, -10) m 가 됩니다.

조건 3, INS 는 무게중심과 축이 일치합니다. INS 좌표계가 동체 좌표계와 같아져서 변환 단계 하나가 통째로 생략 됩니다.

조건 4, 입력은 안테나 기준 거리 · 방위각 · 고각입니다. 체인의 출발점이 안테나 극좌표로 확정됩니다.

조건 5, 처리는 로컬 좌표계에서 합니다. 체인의 중간 목적지가 NED 로 확정됩니다.

조건 6, 출력은 위 · 경 · 고도와 안테나 극좌표 둘 다입니다. 정변환뿐 아니라 역변환도 필요합니다.

조건 4, 5, 6 이 출발점, 중간점, 도착점을 이미 지정합니다. 설계자가 정할 것은 그 사이를 어떻게 이을 것인가입니다.

단계별 좌표계 선정과 그 이유

단계	좌표계	왜 이것인가
0	안테나 — 극좌표	레이다가 물리적으로 잴 수 있는 것이 거리와 두 각도뿐
1	안테나 — 직교좌표	회전행렬은 벡터에만 곱할 수 있다
2	동체 (Body)	안테나·INS·무장이 만나는 유일한 공통 기준. 배를 거치지 않고 바깥으로 나갈 방법이 없다
3	로컬 (NED)	배가 흔들려도 이 좌표계는 흔들리지 않는다. ENU 가 아닌 NED 인 이유는 동체 FRD 와 축 의미가 맞아서
4	ECEF	로컬에서 위경도로 곧장 갈 수 없다 — 다리 역할
5	LLA	전시기·통제제어가 지도 위에 표시하는 형식

쓰지 않은 좌표계

INS

③ 에 의해 동체와 일치. 항등행렬을 곱하는 셈

UV

빔 조향은 안테나 내부의 일 — 데이터 처리 체인 밖

ECI

관성 동역학 전파가 필요할 때만 (위성·탄도탄)

ENU

NED 와 정보량이 같다 — 하나로 통일

발표 대본

설계 · 약 60초

단계별로 어떤 좌표계를 골랐고 왜 골랐는지입니다.

0단계 안테나 극좌표는 레이다가 물리적으로 잴 수 있는 것이 거리와 두 각도뿐이기 때문입니다. 1단계 안테나 직교좌표는 회전행렬이 벡터에만 곱해지기 때문입니다.

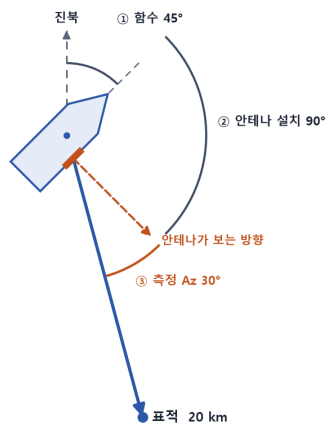
2단계 동체는 안테나 · INS · 무장이 만나는 유일한 공통 기준이고, 배를 거치지 않고 바깥으로 나갈 방법이 없기 때문입니다.

3단계 로컬 NED 는 배가 흔들려도 흔들리지 않기 때문입니다. ENU 가 아닌 NED 인 이유는 동체 FRD 와 축 의미가 맞아서입니다.

4단계 ECEF 는 로컬에서 위경도로 곧장 갈 수 없어서 다리 역할이고, 5단계 LLA 는 전시기와 통제제어가 지도에 표시하는 형식입니다.

쓰지 않은 좌표계도 이유가 있습니다. INS 는 조건 3 에 의해 동체와 일치해서 항등행렬을 곱하는 셈이고, UV 는 빔 조향이라 데이터 처리 체인 밖이고, ECI 는 관성 동역학 전파가 필요할 때만 쓰고, ENU 는 NED 와 정보량이 같아서 하나로 통일했습니다.

배가 기울지 않았다면 — 방위각은 그냥 더해진다



$$\begin{array}{ccccccc} \textcircled{1} & 45^\circ & + & \textcircled{2} & 90^\circ & + & \textcircled{3} & 30^\circ \\ & \text{함수} & & & \text{안테나 설치} & & & \text{측정 Az} \\ & & & & & & = & 165^\circ \end{array}$$

세 번 연달아 오른쪽으로 돌았을 뿐이니, 직관적으로도 당연합니다.

실제 계산값 165.074° — 차이 0.074° 는 레버암 30 m 때문

그런데 왜 굳이 행렬을 쓸까?

roll 이나 pitch 가 조금이라도 있으면 이 덧셈은 즉시 깨집니다.

세 축의 회전이 서로 섞이기 때문입니다.

발표 대본

설계 · 약 40초

직관 확인입니다. 함수 45도, 안테나 설치 90도, 측정 방위각 30도를 더하면 165도입니다. 세 번 연달아 오른쪽으로 돌았을 뿐이니 당연합니다. 실제 계산값은 165.074도이고, 차이 0.074도는 레버암 30 m 때문입니다.

그런데 왜 굳이 행렬을 쓰느냐 하면, roll 이나 pitch 가 조금이라도 있으면 이 덧셈은 즉시 깨지기 때문입니다. 세 축의 회전이 서로 섞입니다. 뒤의 확인 실험에서 그 숫자를 보여드리겠습니다.

단계별 변환 ①~③ 안테나 → 동체 → 로컬

1 안테나 극좌표 → 직교좌표

표현 변경 (회전·이동 없음)

$$\begin{aligned} x &= R \cdot \cos(E1) \cdot \cos(Az) = +17\ 320.5081 \\ y &= R \cdot \cos(E1) \cdot \sin(Az) = +10\ 000.0000 \\ z &= -R \cdot \sin(E1) = 0.0000 \end{aligned}$$

2 안테나 → 동체

회전 + 평행이동

$$\begin{aligned} C_{\text{body} \leftarrow \text{ant}} &= R_z(90^\circ) \cdot R_y(0^\circ) \\ p_{\text{body}} &= C \times p_{\text{ant}} + (-30, 0, -10) \\ &= (-10\ 030.0, +17\ 320.5, -10.0) \end{aligned}$$

3 동체 → 로컬 (NED)

회전만 (원점이 같음)

$$\begin{aligned} C_{\text{ned} \leftarrow \text{body}} &= R_z(45^\circ) \cdot R_y(0^\circ) \cdot R_x(0^\circ) \\ p_{\text{ned}} &= (N\ -19\ 339.73, E\ +5\ 155.17, D\ -10.0) \\ \text{진북 기준 방위} &165.074^\circ \end{aligned}$$

여기가 데이터 처리 구간 — 추적·필터링은 로컬 좌표계에서 수행합니다

발표 대본

설계 · 약 55초

예제값으로 앞 세 단계를 따라가 보겠습니다.

1단계, 극좌표를 직교좌표로. 표현만 바꿉니다. $x = +17,320.5081$, $y = +10,000$, $z = 0$ 입니다.

2단계, 안테나에서 동체로. 회전과 평행이동입니다. $C_{\text{body} \leftarrow \text{ant}}$ 는 $R_z(90^\circ) \cdot R_y(0^\circ)$ 이고, 여기에 곱한 뒤 $(-30, 0, -10)$ 을 더하면 $(-10,030.0, +17,320.5, -10.0)$ 입니다. x 와 y 가 자리를 바꾸고 부호가 뒤집힌 것이 보입니다.

3단계, 동체에서 NED 로. 원점이 같으니 회전만입니다. $C_{\text{ned} \leftarrow \text{body}}$ 는 $R_z(45^\circ) \cdot R_y(0^\circ) \cdot R_x(0^\circ)$ 이고, 결과는 $N\ -19,339.73$, $E\ +5,155.17$, $D\ -10.0$ 입니다. 진북 기준 방위 165.074° 도입니다.

여기가 데이터 처리 구간입니다. 추적과 필터링은 이 로컬 좌표계에서 수행합니다.

단계별 변환 ④~⑤ 로컬 → ECEF → 위경도

4 로컬 → ECEF

회전 + 평행이동

- ① 배의 위치를 ECEF 로
- ② 로컬 벡터를 지구 방향으로 회전
(회전량은 배의 위도·경도가 결정)
- ③ 더한다

5 ECEF → LLA

위도 · 고도 반복 계산

닭-달걀 관계라 딱 떨어지는 공식이 없다.
위도를 알아야 곡률반경 N 을 구하는데
 N 을 알아야 위도를 구할 수 있기 때문.
→ 구로 본 위도에서 출발, 값이 안 변할 때까지 반복

표적 LLA = 36.233700252°N 127.364344961°E 고도 41.4952 m

왜 고도가 10 m 가 아니라 41.5 m 인가?

레이다는 수평($\theta = 0^\circ$)으로 쏘고 안테나는 10 m 높이니 "표적 고도 10 m" 일 것 같습니다.
그런데 빔은 직진하는데 바다 표면은 멀어질수록 아래로 휘어 내려갑니다. 20 km 지점에서 그 차이가 31.5 m, $10 + 31.5 = 41.5$ m.

발표 대본

설계 · 약 70초

4단계, 로컬에서 ECEF 로. 회전과 평행이동입니다. 배의 위치를 ECEF 로 바꾸고, 로컬 벡터를 지구 방향으로 회전하고, 둘을 더합니다. 회전량은 배의 위도 · 경도가 결정합니다.

5단계, ECEF 에서 LLA 로. 위도와 고도가 서로 물려 있어서 딱 떨어지는 공식이 없습니다. 위도를 알아야 곡률반경 N 을 구하는데 N 을 알아야 위도를 구할 수 있기 때문입니다. 그래서 지구를 구로 본 위도에서 출발해서 값이 안 변할 때까지 반복합니다. 보통 5 ~ 7회면 수렴합니다.

표적 LLA 는 36.233700252°N, 127.364344961°E, 고도 41.4952 m 입니다.

여기서 질문이 하나 나옵니다. 왜 고도가 10 m 가 아니라 41.5 m 인가. 레이다는 수평, $\theta 0^\circ$ 로 쏘고 안테나는 10 m 높이니 표적 고도 10 m 일 것 같습니다. 그런데 빔은 직진하는데 바다 표면은 멀어질수록 아래로 휘어 내려갑니다. 20 km 지점에서 그 차이가 31.5 m 이고, 10 더하기 31.5 가 41.5 m 입니다. 앞에서 본 로컬 평면의 한계가 바로 이 숫자입니다.

되돌아오기 — 역변환

조건 ⑥ 은 결과를 두 가지 형태로 요구합니다

역변환은 새로 배울 것이 없습니다

① 회전은 전치 (transpose) ② 평행이동은 빼기 ③ 순서는 거꾸로

```
표적 LLA (36.233700°N, 127.364345°E, 41.4952 m)
→ ECEF          // 정변환과 같은 공식
→ 로컬 NED      C_ned←e × (표적 - 배)
→ 동체          C_ned←body^T × p_ned          // 전치
→ 안테나        C_body←ant^T × (p_body - 레버암) // 전치 + 빼기
→ 극좌표        R = √(x²+y²+z²), Az = atan2(y, x), El = atan2(-z, √(x²+y²))
```

결과 : R = 20 000.000000 m Az = 30.000000° El = 0.000000°

입력값과 정확히 같은 값이 돌아왔습니다. 왕복오차 $\Delta R = 0.0$ m, $\Delta Az = 1.8e-12^\circ$ — 컴퓨터 반올림 수준입니다.

부호 하나, 전치 방향 하나만 틀려도 이 왕복 시험은 즉시 실패합니다 — 그래서 가장 강력한 검증 수단이 됩니다.

발표 대본

설계 · 약 60초

조건 6 은 결과를 두 가지 형태로 요구합니다. 그래서 역변환이 필요한데, 새로 배울 것이 없습니다. 회전은 전치, 평행이동은 빼기, 순서는 거꾸로입니다.

표적 LLA 에서 출발해 ECEF 로 가는 것은 정변환과 같은 공식이고, 로컬 NED 는 표적에서 배를 뺀 것을 회전하고, 동체는 $C_{ned} \leftarrow body$ 의 전치를 곱하고, 안테나는 레버암을 뺀 뒤 $C_{body} \leftarrow ant$ 의 전치를 곱하고, 마지막에 극좌표로 돌아옵니다. R 은 제곱합의 제곱근, Az 는 $atan2(y, x)$, El 은 $atan2(-z, \text{수평거리})$ 입니다.

결과는 $R = 20,000.000000$ m, $Az = 30.000000^\circ$, $El = 0.000000^\circ$ 입니다. 입력값과 정확히 같은 값이 돌아왔습니다. 왕복오차는 ΔR 0.0 m, ΔAz $1.8e-12^\circ$ 로 컴퓨터 반올림 수준입니다.

부호 하나, 전치 방향 하나만 틀려도 이 왕복 시험은 즉시 실패합니다. 그래서 가장 강력한 검증 수단이 됩니다.

Part 3

C 로 짜서 확인하기

좌표변환은 "적당히 그럴듯한 값"이 나오기 때문에 눈으로는 검증되지 않습니다

Visual Studio 2022 · C17 · 경고 0건 · 왕복 오차 1e-9 m 미만

발표 대본

섹션 · 약 20초

세 번째 주제입니다. 좌표변환은 "적당히 그럴듯한 값" 이 나오기 때문에 눈으로는 검증되지 않습니다. 그래서 C 로 짜서 숫자로 확인했습니다. Visual Studio 2022, C17 로 경고 없이 빌드했고, 되돌린 값이 입력과 반올림 수준까지 맞는 것을 확인했습니다.

구현 — 애초에 실수하기 어렵게 만들기

좌표변환 버그의 특징은 컴파일도 되고 값도 그럴듯해 보인다는 것입니다

①

함수 이름에 방향을 박는다

```
f_BodyToNed() / f_NedToBody()
f_AntToBody() / f_BodyToAnt() / f_EcefToLla()
```

②

회전은 Rx·Ry·Rz 곱으로만 만든다

```
Matrix_Product3(&Rz, &Ry, &Rx) / 역변환은 Matrix_Transpose()
행렬 연산은 참고 소스 matrixCalLib 그대로. 원소를 손으로 안 쓴다
```

③

단위는 안에서 하나로 통일한다

rad · m 만 쓴다. deg 는 DEG2RAD() / RAD2DEG() 로 입출력에서만
중간에 deg 가 섞이면 값이 그럴듯해서 더 못 잡는다

검증 — 갔다가 돌아오면 원래 값이 나와야 합니다

왕복 시험	측정값 → LLA → 측정값. 콘솔과 UI 가 입력과의 차이를 같이 찍음	부호 · 전치 방향 · 순서 실수
예제값 손계산	동체 좌표는 Rz(90°) 로 돌리고 오프셋을 더하면 손으로 나옴	레버암 부호, 설치 방향
자세 바꿔 가며	UI 슬라이더로 roll · pitch · yaw 를 다르게 놓고 왕복 확인	대칭 자세에선 안 드러나는 전치 오류

발표 대본

구현 · 약 80초

좌표변환 버그의 특징은 컴파일도 되고 값도 그럴듯해 보인다는 것입니다. 그래서 애초에 실수하기 어렵게 세 가지 원칙을 두었습니다.

첫째, 함수 이름에 방향을 박았습니다. f_BodyToNed 와 f_NedToBody, f_AntToBody 와 f_BodyToAnt 처럼 어디서 어디로 가는지가 이름에 있습니다. 인자 순서를 헷갈릴 자리가 없습니다.

둘째, 회전은 Rx, Ry, Rz 곱으로만 만들었습니다. 행렬 연산은 참고 소스로 받은 matrixCalLib 을 그대로 썼습니다. 위치는 3x1, 회전은 3x3 matrix 로 두고, 동체에서 NED 로 가는 회전은 Matrix_Product3 에 Rz, Ry, Rx 를 넣어 만들고, 역변환은 Matrix_Transpose 하나입니다. 행렬 원소를 손으로 쓰지 않으니 부호 실수가 끼어들 자리가 없습니다.

셋째, 단위는 안에서 하나로 통일했습니다. 라이브러리 안에서는 라디안과 미터만 쓰고, 도는 DEG2RAD, RAD2DEG 로 입출력에서만 바꿉니다. 중간에 도가 섞이면 값이 그럴듯해서 더 못 잡습니다.

검증 원리는 하나입니다. 갔다가 돌아오면 원래 값이 나와야 합니다. 콘솔과 UI 가 정변환 뒤 역변환한 값을 입력과 비교해서 차이를 같이 찍고, 예제값의 동체 좌표는 손으로 계산한 값과 맞춰 봤습니다. 전치 방향이 뒤바뀐 실수는 대칭적인 자세에서는 드러나지 않기 때문에, UI 슬라이더로 roll, pitch, yaw 를 전부 다른 값으로 놓고 왕복오차가 그대로 반올림 수준인지 확인합니다.

최종 출력값

조건 ⑥ 이 요구한 두 가지 형태

입력 : R 20 km · Az 30° · El 0° | 플랫폼 36.408°N 127.307°E 0 m (정지) | 자세 roll 0° · yaw 45° · pitch 0°

[1] 위 / 경 / 고도

위도 **36.233700252 °N**

경도 **127.364344961 °E**

고도 **41.4952 m**

WGS-84 타원체고

[2] 안테나 극좌표

시선거리 **20 000.0000 m** 0.0 m

방위각 **30.000000 °** 1.8e-12 °

고각 **0.000000 °** 1.7e-12 °

오른쪽은 입력값과의 차이 (왕복오차)

역변환이 입력값과 컴퓨터 반올림 수준까지 일치 — 변환 체인 전체가 서로 정확히 역관계임을 보여줍니다

1e-12 °

왕복 각도 오차

0 건

컴파일 경고

< 1e-9 m

왕복 거리 오차

발표 대본

결과 · 약 40초

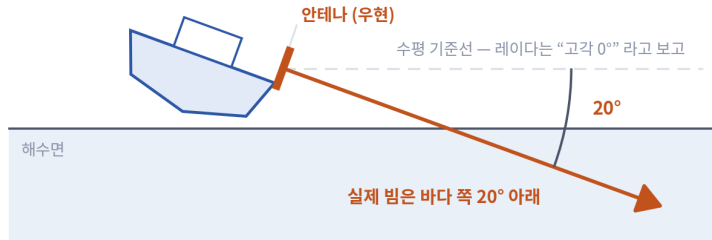
조건 6 이 요구한 두 가지 형태입니다.

첫째, 위 · 경 · 고도. 위도 36.233700252°N, 경도 127.364344961°E, 고도 41.4952 m 이고 WGS-84 타원체 고입니다.

둘째, 안테나 극좌표. 시선거리 20,000.0000 m, 방위각 30.000000°, 고각 0.000000° 입니다. 오른쪽 작은 숫자가 입력값과의 차이, 즉 왕복오차이고 거리는 0.0 m, 각도는 10의 -12승 도 수준입니다.

역변환이 입력값과 컴퓨터 반올림 수준까지 일치합니다. 변환 체인 전체가 서로 정확히 역관계라는 뜻입니다. 왕복 각도 오차 1e-12도, 컴파일 경고 0건, 왕복 거리 오차 1e-9 m 미만입니다.

확인 실험 — 설계 판단이 실제로 근거가 있는가



실험 B 배가 기울면

같은 측정값 (Az 30°, El 0°) 에 자세만 바꿔 넣은 결과

roll	실제 고각
0°	+0.03°
10°	-8.61°
20°	-17.19°

레이더는 여전히 "고각 0도" 라고 보고합니다. 그건 안테나 기준일 뿐입니다. roll 20° 면 20 km 앞 고도가 5.9 km 어긋난다.

실험 A 레버암을 빼먹으면

수평으로 정확히 30 m, 고도로 10 m 어긋난다. 거리가 멀어져도 이 오차는 줄지 않는다.

실험 C 지구 곡률

5 km 2 m · 20 km 31 m · 100 km 786 m. 거리의 제곱에 비례해 커진다.

각도를 더하는 방식으로는 이 효과를 만들 수 없습니다 — 해상 플랫폼에서 회전형렬은 선택이 아니라 필수입니다.

발표 대본

실험 · 약 60초

설계 판단이 실제로 근거가 있는지 세 가지 실험으로 확인했습니다.

실험 A, 레버암을 빼먹으면 수평으로 정확히 30 m, 고도로 10 m 어긋납니다. 거리가 멀어져도 이 오차는 줄지 않습니다.

실험 B, 배가 기울면. 같은 측정값 Az 30도, El 0도에 자세만 바꿔 넣었습니다. roll 0도에서 실제 고각 +0.03도, 10도에서 -8.61도, 20도에서 -17.19도입니다. 레이더는 여전히 "고각 0도" 라고 보고하지만 그건 안테나 기준일 뿐이고, roll 20도면 20 km 앞에서 고도가 약 5.9 km 어긋납니다.

실험 C, 지구 곡률. 5 km 에서 2 m, 20 km 에서 31 m, 100 km 에서 786 m 로 거리의 제곱에 비례해 커집니다.

각도를 더하는 방식으로는 이 효과를 만들 수 없습니다. 해상 플랫폼에서 회전형렬은 선택이 아니라 필수입니다.

UI 로 확인 — CoordUI

같은 CoordCore 를 물고, 입력을 바꾸면 바로 다시 계산해서 단계별 값을 보여줍니다

프로그램 구성

CoordCore
좌표변환 정적 라이브러리 (C)
coord_frames + 참고 소스 matrixCalcLib

radar_coord
콘솔, 과제 3.1 입력값으로 단계별 값 출력
왕복오차까지 같이 찍음

CoordUI
MFC 다이얼로그, 오른쪽 화면
변환 로직 없이 f_ 함수만 부름

실행 프로젝트 둘 다 CoordCore 를
ProjectReference 로 물고 있음

CoordFrames

측정값 R [m] / Az / El [deg]

20000.0 30.000 0.000

플랫폼 위치 lat / lon [deg] / alt [m]

36.408 127.307 0.0

플랫폼 자세 [deg] roll / pitch / yaw

roll 0.0

pitch 0.0

yaw 45.0

안테나 설치 az / tilt [deg] / offset [m]

90.000 0.000 -30, 0, -10

예제값 복원

입력 각도: 도 / 변환 계산: 라디안, m

단계별 결과

단계	x / N / X	y / E / Y	z / D / Z
1) 극좌표 -> 안테나 직교	17320.5	10000.0	0.0
2) 안테나 -> 동체	-10030.0	17320.5	-10.0
3) 동체 -> NED	-19339.7	5155.2	-10.0
4) NED -> ECEF	-3125892.5	4093775.0	3749164.2
5) ECEF -> LLA	36.233700	127.364345	41.4952
역변환 -> 극좌표	20000.0	30.000000	0.000000

최종 출력

위/경/고도 36.233700252 N 127.364344961 E 41.4952 m
 안테나 극좌표 R 20000.0000 m Az 30.000000° El 0.000000°
 왕복오차 dR 0.0 m dAz 1.8e-12° dEl 1.7e-12°

roll 슬라이더를 돌리면 앞 장 실험 B 의 고각 변화가 그 자리에서 보입니다. 발표 중에 직접 돌려 봅니다.

발표 대본

UI · 약 45초

C 코드는 프로젝트 세 개로 나뉘었습니다. 변환 함수와 참고 소스 matrixCalcLib 은 CoordCore 라는 정적 라이브러리에 두고, 콘솔 radar_coord 와 MFC 다이얼로그 CoordUI 가 같은 라이브러리를 씁니다. 콘솔은 과제 3.1 값과 왕복오차를 찍습니다.

CoordUI 는 입력칸에 측정값, 플랫폼 위치와 자세, 안테나 설치값을 넣으면 바로 다시 계산해서 오른쪽 표에 단계별 값을, 아래에 최종 출력 두 가지와 왕복오차를 보여줍니다. 다이얼로그에는 변환 로직이 없고 라이브러리의 f_ 함수만 부릅니다.

자세는 슬라이더로도 바꿀 수 있어서, roll 을 10도, 20도로 돌리면 앞 장 실험 B 의 고각 변화가 그 자리에서 보입니다. 발표 중에 직접 돌려 보겠습니다.

정리

- 1 좌표계는 "어디에 서서 어느 방향을 기준으로 재는가" 이고, 좌표변환은 그 기준을 맞추는 계산이다
- 2 변환의 재료는 회전과 평행이동 둘뿐이다
- 3 순서는 안테나 → 동체 → 로컬 → ECEF → 위경도. 건너뛴 수 없다
- 4 회전은 설치 방위와 INS 자세·위경도가, 평행이동은 레버암과 배의 위치가 공급한다
- 5 돌아올 때는 전치하고, 빼고, 순서를 뒤집는다. 왕복이 맞으면 구현이 맞은 것이다
- 6 배가 기울면 각도 덧셈은 깨진다 — 회전행렬은 필수다
- 7 로컬 평면 좌표를 고도로 그대로 쓰면 안 된다. 지구는 둥글다

발표 대본

정리 · 약 50초

일곱 가지로 정리하겠습니다.

하나, 좌표계는 "어디에 서서 어느 방향을 기준으로 재는가" 이고 좌표변환은 그 기준을 맞추는 계산입니다. 둘, 변환의 재료는 회전과 평행이동 둘뿐입니다. 셋, 순서는 안테나, 동체, 로컬, ECEF, 위경도이고 건너뛴 수 없습니다. 넷, 회전은 설치 방위와 INS 의 자세 · 위경도가, 평행이동은 레버암과 배의 위치가 공급합니다. 다섯, 돌아올 때는 전치하고 빼고 순서를 뒤집습니다. 왕복이 맞으면 구현이 맞은 것입니다. 여섯, 배가 기울면 각도 덧셈은 깨지고 회전행렬은 필수입니다. 일곱, 로컬 평면 좌표를 고도로 그대로 쓰면 안 됩니다. 지구는 둥글다.

감사합니다

질문 환영합니다

측정

안테나

동체

NED

ECEF

LLA

코드 · 문서 : radar-sw-study / Part2_CoordFrames

발표 대본

마무리 · 약 10초

이상으로 발표를 마치겠습니다. 코드와 문서는 radar-sw-study 저장소의 Part2_CoordFrames 에 있습니다. 질문 환영합니다.